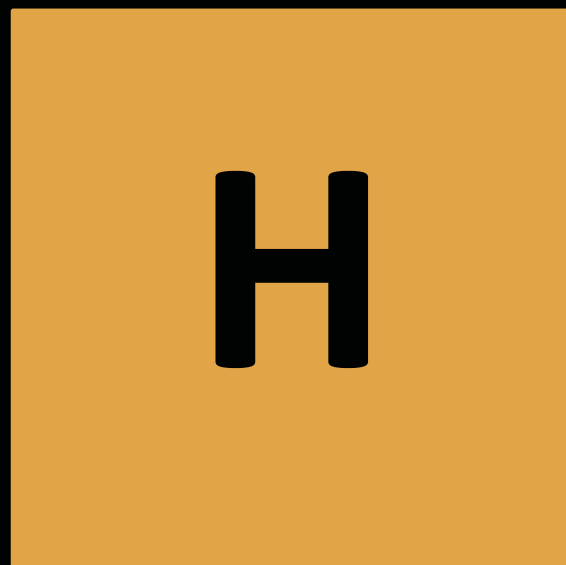




HERMES

Car Rental Platform

By:

*Asylbekov Islambek &
Anarbekov Azimbek*

TYPE

Mobile + Backend Application

MOBILE

Flutter & Dart

BACKEND

Java Spring Boot

DATABASE

PostgreSQL

YEAR

2026

THE PROBLEM

Renting a car today is painful — for both sides

FOR RENTERS

- ! Agency markups add 30–50% on top of actual car cost
- ! Booking requires physical presence and paper contracts
- ! No way to see real availability — must call or visit in person
- ! Zero transparency: hidden insurance fees, fuel policies

FOR CAR OWNERS

- ! Idle car sitting in a parking lot earns nothing
- ! No direct channel to find trusted renters nearby
- ! Listing on agencies means losing most of the profit
- ! No app to manage bookings, dates, or pricing on the go

THE SOLUTION

One app. Two roles. Zero middlemen.

01 Mobile-First

Full booking flow from discovery to confirmation — done entirely on your phone in under 3 minutes.

02 Peer-to-Peer

Owners list their car directly. Renters book directly. Hermes handles auth, payments and trust.

03 Role System

Two roles: CLIENT books cars, OWNER lists them. Clear separation of permissions and capabilities.

04 Transparent Pricing

Price set by the owner, shown upfront. Balance deducted at booking. No surprise fees.

05 Real Photos

Owners upload actual car photos from their device camera. What you see is what you get.

06 Secure Auth

JWT tokens protect every endpoint. Spring Security enforces roles server-side, every request.

KEY FEATURES

01 Car Catalog

All listed vehicles with photos, specs, fuel type, capacity and live availability.

03 Date-Based Booking

Pick exact pickup and return dates. Conflict detection prevents double bookings.

05 Wallet & Balance

Users top up in-app balance. Payment deducted on booking confirmation.

07 Dashboard

Overview of listed cars, incoming bookings in one screen.

02 Smart Filters

Narrow results by car class, brand, price range, seat count and date range.

04 Add Car

Multi-field form: brand, model, year, fuel type, capacity, price, real camera photo.

06 Profile Management

Edit personal data, view rental history, manage role and account settings.

08 Photo Upload

MultipartFile support — owners upload real images directly from device camera.

TECH STACK

FRONTEND

Flutter

Cross-platform UI framework — one codebase for iOS and Android

Dart

Strongly-typed language powering all Flutter app logic

Dio

HTTP client for REST calls, multipart uploads and JWT headers

BACKEND

Java Spring Boot

Production REST API with dependency injection and layered architecture

Spring Security

JWT-based stateless auth, role enforcement on every endpoint

Spring Data JPA

Repository abstraction over PostgreSQL via Hibernate ORM

INFRASTRUCTURE

PostgreSQL

Relational database for users, cars, bookings and transactions

Ngrok

Localhost tunnel — exposes Spring Boot to the physical device during dev

Postman

API testing and endpoint verification during development

API FLOW — Adding a Car

Tracing one request from the user's tap to the database and back

1

User taps Submit

Flutter collects brand, model, price, fuelType, description, image (File) from AddCarPage form.

2

Dio builds request

Dio creates multipart/form-data POST /api/cars with Authorization: Bearer <JWT> header attached.

3

Security Filter

Spring Security intercepts every request, decodes JWT, checks ROLE_OWNER — returns 403 if missing.

4

Controller receives DTO

@PostMapping /api/cars receives CarRequest DTO with all fields. Passes to CarService layer.

5

Service validates & saves

CarService validates data, maps to Car entity, calls CarRepository.save() → PostgreSQL INSERT.

6

Response to Flutter

201 Created + CarResponse JSON returned. Flutter updates catalog list UI — new car appears instantly.

POST /api/cars

```
{  
  
  brand:  "Toyota",  
  model:  "Camry",  
  year:   2022,  
  price:  5000,  
  fuel:   "Petrol",  
  seats:  5,  
  desc:   "...",  
  image:  <binary>  
}
```

← 201 Created

CHALLENGES WE SOLVED

Real problems encountered during development — and how we fixed them

403

Forbidden on Owner endpoints

CLIENT role users couldn't trigger the add-car endpoint from the UI, creating confusing silent failures.

✓ Allowing for CLIENT role to add their own car, so they let other users to rent them

404

Endpoint Not Found

Flutter's base URL and Spring Boot's @RequestMapping paths were misaligned — one had a trailing slash, the other didn't. Requests vanished silently.

✓ Standardised base URL in Dio config. Used Postman to verify each path independently before connecting Flutter. All routes now tested before wiring to UI.

IMG

Image upload through Ngrok

Multipart image uploads worked in Postman but failed completely when routed through the Ngrok tunnel — the boundary header was being stripped.

✓ Added explicit Content-Type: multipart/form-data with boundary in Dio options. Confirmed with raw curl through the tunnel. Uploads now work on real device.

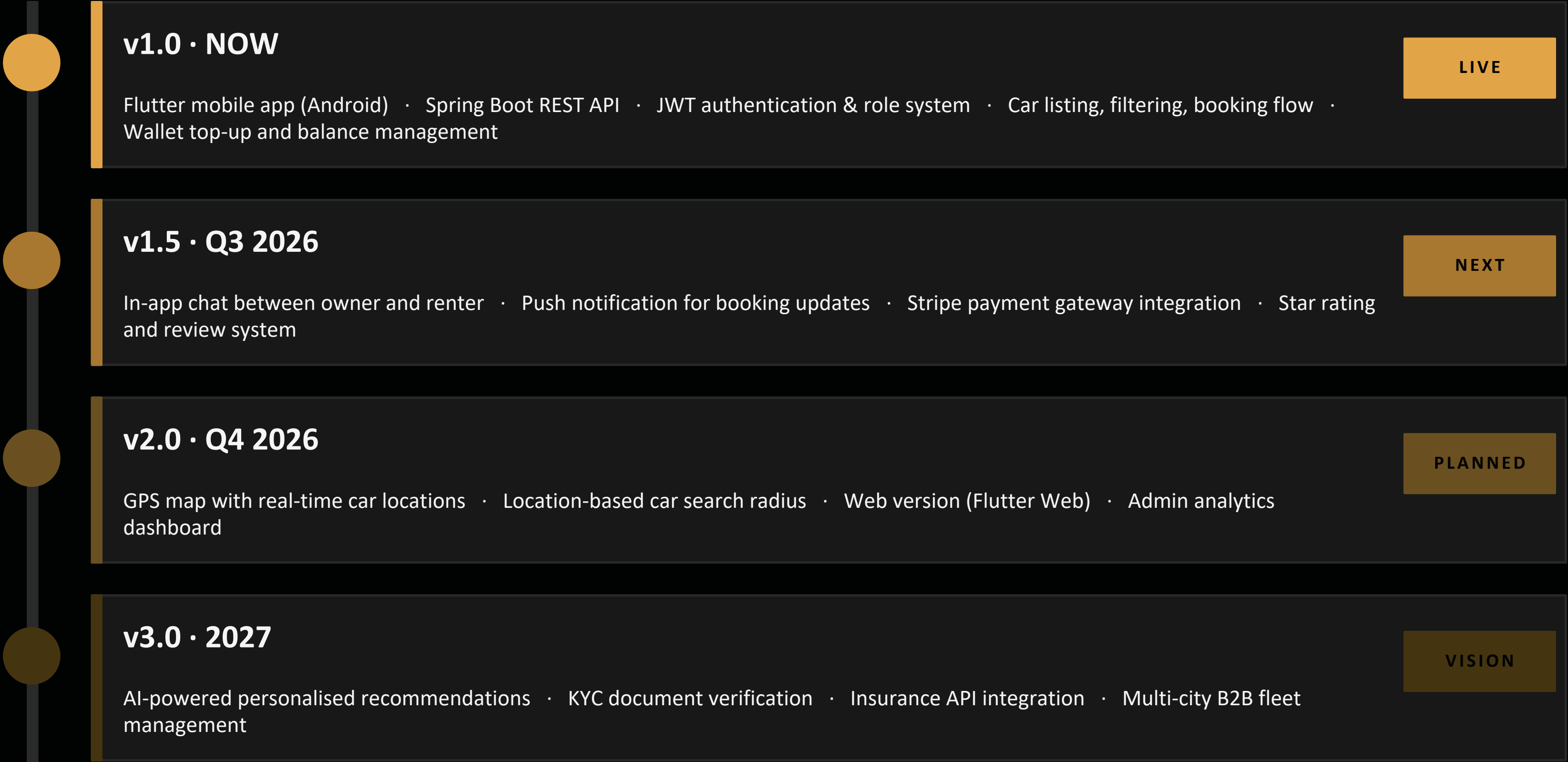
DTO

Java ↔ Dart type mismatch

Java DTO used Integer for price; Dart model declared it as double. JSON parsing threw a silent cast exception and the car was never saved.

✓ Aligned all numeric types: Integer in Java DTO, int in Dart model. Added explicit type annotations on both sides and tested with edge values (0, nulls, large numbers).

ROADMAP



H

Thank You

Questions & Discussion

WHAT WE BUILT

Cross-platform Flutter app for Android — browse, filter, book, manage

Java Spring Boot REST API with JWT auth, role-based access, and PostgreSQL

Two-sided marketplace: CLIENT rents, OWNER lists — protected by Spring Security

Real photo upload, wallet system, full booking flow from browse to confirmation